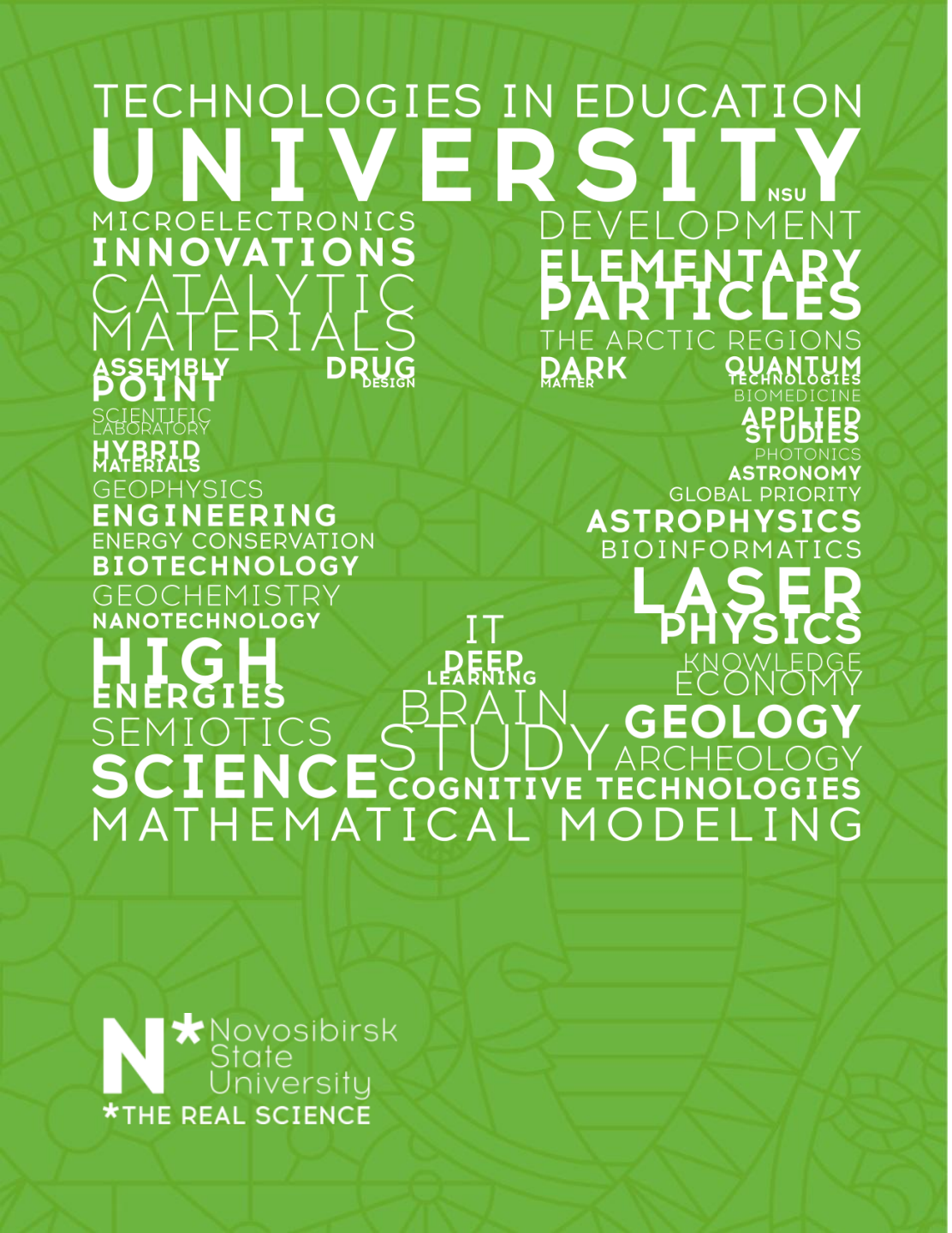


Базовые методы ИИ
Семинар 9

Глушенко Андрей Валерьевич
ФФ НГУ

Практические аспекты

CART

Случайные деревья

Случайный лес

Ограничения ID3 (почему появились C4.5 и CART)

- Только категориальные признаки: В оригинальном ID3 нельзя использовать числа (например, «Вес 75.5 кг»), только категории («Высокий», «Низкий»).
- Склонность к переобучению: ID3 строит очень глубокие деревья, которые идеально знают обучающую выборку, но ошибаются на новых данных.
- Проблема многозначных признаков: Если у вас есть признак «ID клиента» (где каждое значение уникально), ID3 выберет его как идеальный, так как в каждой ветке будет по 1 чистому объекту. Но это бесполезно для предсказаний.

Критерии качества решающих деревьев

Для построения эффективного решающего дерева необходимо на каждом этапе выбирать такое разбиение данных, которое максимально увеличивает «чистоту» получаемых групп. Для этого используются **критерии информативности** (меры неопределенности), которые оценивают качество распределения классов в узле.

Энтропийный критерий (Entropy)

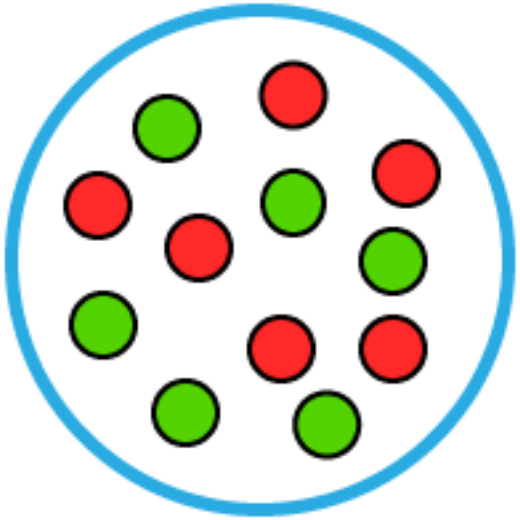
Основан на понятии информационной энтропии Шеннона и измеряет степень хаоса или неопределенности в данных.

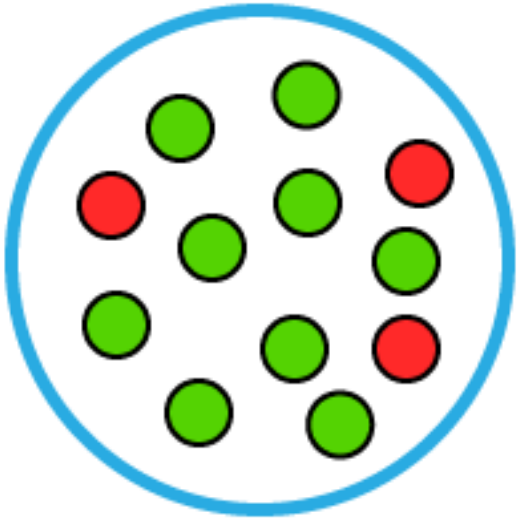
$$H = -\sum_{i=1}^{k} p_i \log_2 p_i \,,\ p_i \ - \text{доля объектов класса } i$$

- 0 — это чистый узел. Значение 1 соответствует максимальному беспорядку (для двух классов) (состояние узла, в котором объекты разных классов перемешаны в **равных пропорциях**.).
- Более чувствителен к изменениям в распределении классов, чем критерий Джини, но требует больше вычислительных ресурсов. Используется в алгоритмах ID3 и C4.5.

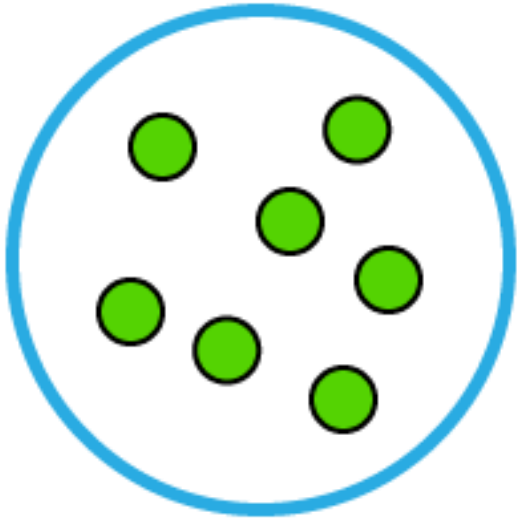
Беспорядок

Максимальный беспорядок





Чистый узел



Критерий Джини (Gini Impurity)

Индекс Джини измеряет вероятность того, что случайно выбранный элемент из узла будет классифицирован неправильно, если назначить ему метку случайным образом в соответствии с распределением классов в этом узле.

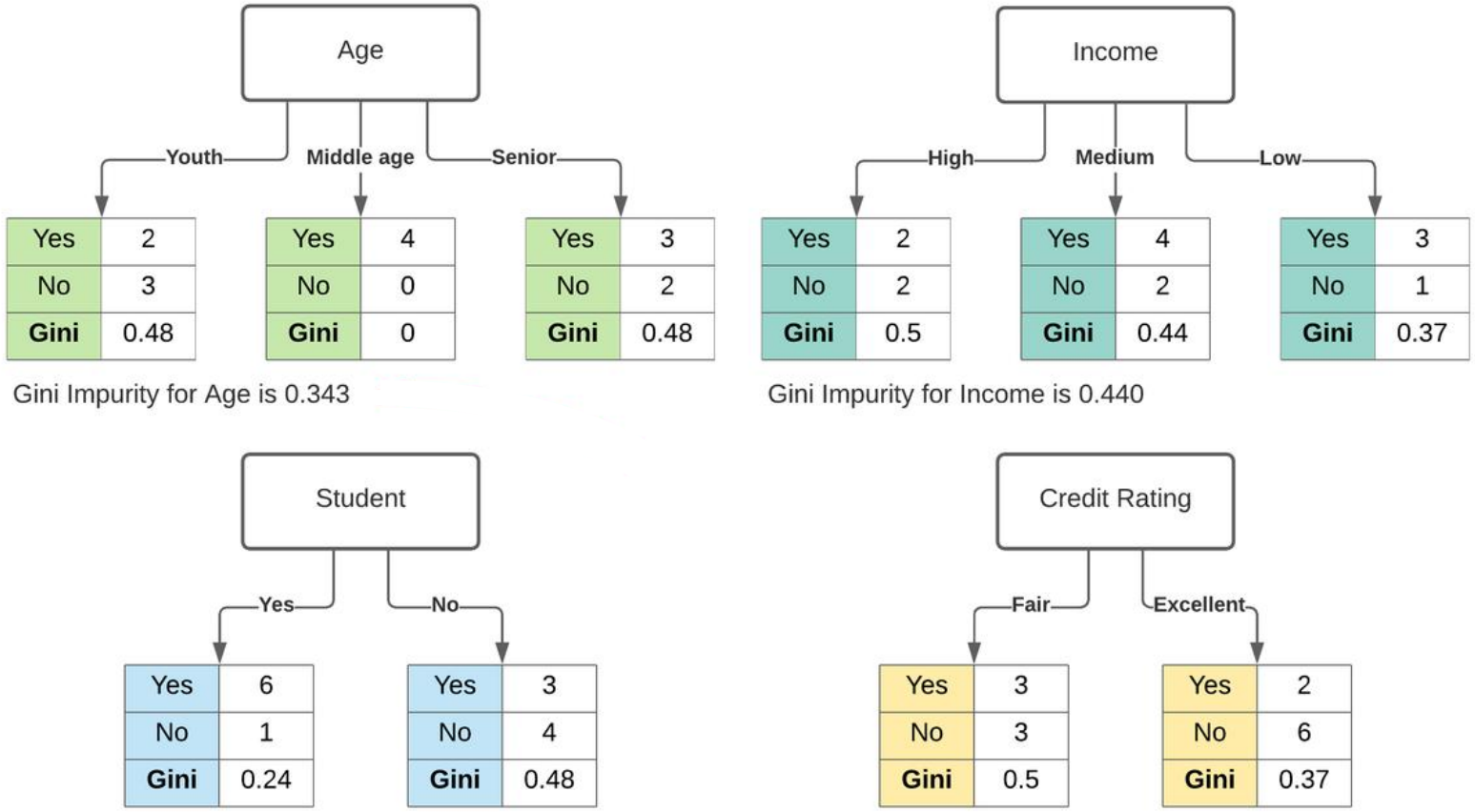
$$G = 1 - \sum_{i=1}^{k} {p_i}^2,\ p_i\ - \text{доля объектов класса } i$$

- Значение 0 означает идеальную чистоту (все объекты одного класса). Максимальное значение (0.5 для двух классов) означает равномерное распределение.
- Вычисляется быстрее энтропии, так как не требует логарифмирования. Используется в алгоритме CART.

Критерий Джини (Gini Impurity)

$$G = 1 - \sum_{i=1}^{k} {p_i}^2,\ p_i\ - \text{доля объектов класса } i$$

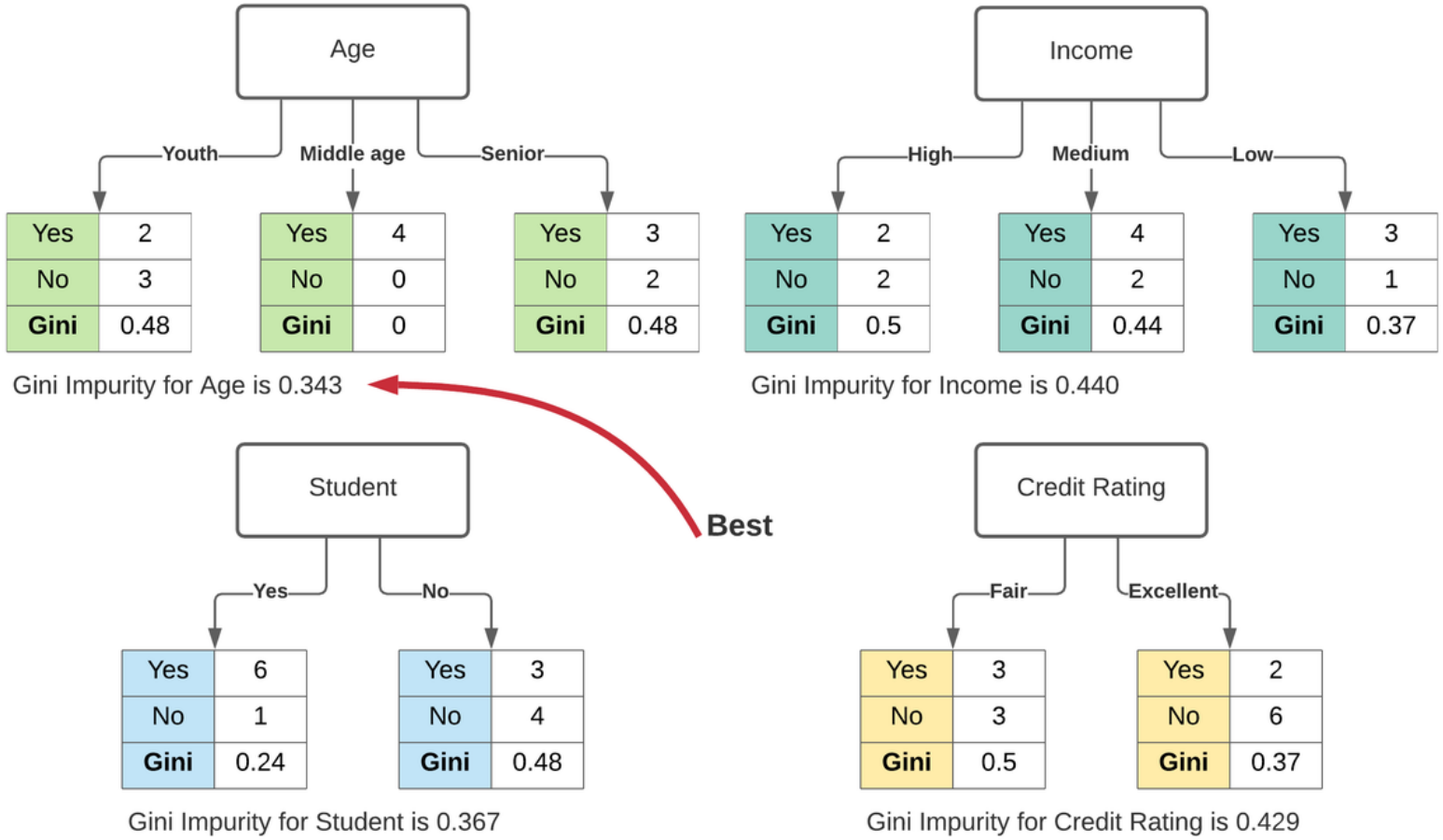
$$G_{split} = \sum_{j=1}^{k} \frac{n_j}{n} \cdot G_j$$



Критерий Джини (Gini Impurity)

$$G = 1 - \sum_{i=1}^{k} {p_i}^2,\ p_i\ - \text{доля объектов класса } i$$

$$G_{split} = \sum_{j=1}^{k} \frac{n_j}{n} \cdot G_j$$



CART (Classification and Regression Trees)

CART (Classification and Regression Trees) — это классический алгоритм построения бинарных деревьев решений. Его главная фишка в том, что он универсален: подходит и для классификации (когда предсказываем категорию), и для регрессии (когда предсказываем число).

CART (Classification and Regression Trees)

1. Начало (Корень): Мы берем весь обучающий набор данных. Это наш «корневой узел».

2. Поиск лучшего разделения:

- Алгоритм перебирает все доступные признаки (например, возраст, доход, рост).
- Для каждого признака он пробует все возможные пороговые значения (например, «Доход > 50к», «Доход > 60к»).
- Цель: найти такую пару «признак + порог», которая лучше всего разделит данные на две группы («левую» и «правую»).

CART (Classification and Regression Trees)

3. Оценка качества (Критерий):

- В **регрессии**: минимизируется среднеквадратичная ошибка (**MSE**). Алгоритм ищет разделение, при котором значения в полученных ветвях будут максимально близки к своему среднему.
- В **классификации**: используется индекс **Джини** (Gini Impurity). Он показывает, насколько «смешаны» классы в узле. Чем меньше индекс, тем более «чистым» (однородным по классам) стал узел.

4. Бинарное разбиение: После выбора лучшего признака данные делятся строго на две части. В CART всегда только две ветки из одного узла.

CART (Classification and Regression Trees)

5. Рекурсия: Эти же шаги повторяются для каждого из двух новых узлов. Процесс идет «вглубь», пока не сработает условие остановки.

6. Остановка: Дерево перестает расти, если:

- Достигнута максимальная глубина.
- В узле осталось слишком мало объектов.
- Дальнейшее разбиение не дает значимого улучшения качества.

7. Финальный прогноз (Листья):

- В регрессии: среднее значение целевой переменной всех объектов в этом листе.
- В классификации: самый часто встречающийся класс (мажоритарный).

Случайные деревья и случайный лес

Случайный лес (Random Forest) — это мощный алгоритм машинного обучения, который объединяет предсказания множества отдельных «решающих деревьев», чтобы получить один более точный и стабильный результат.

Случайные деревья

Случайное дерево решений - основа метода

Представьте дерево как последовательность вопросов, на которые можно ответить «да» или «нет». Например, чтобы решить, пойдет ли человек на прогулку, дерево может спросить: «Идет ли дождь?», а затем «Какая температура?».

Проблема одного дерева: Оно склонно к «переобучению» — слишком сильно подстраивается под обучающие данные и начинает ошибаться на новых примерах.

Случайный лес

Алгоритм решает проблему переобучения, создавая целый «лес» из таких деревьев. Чтобы деревья были разными и независимыми, используются два приема случайности:

Бэггинг (Bagging): Каждое дерево обучается не на всем наборе данных, а на его случайной подвыборке. Это называется «бутстрэп» (bootstrap).

Случайные признаки: При поиске лучшего вопроса для разделения данных (сплита), алгоритм выбирает не из всех возможных характеристик, а только из случайного их подмножества.

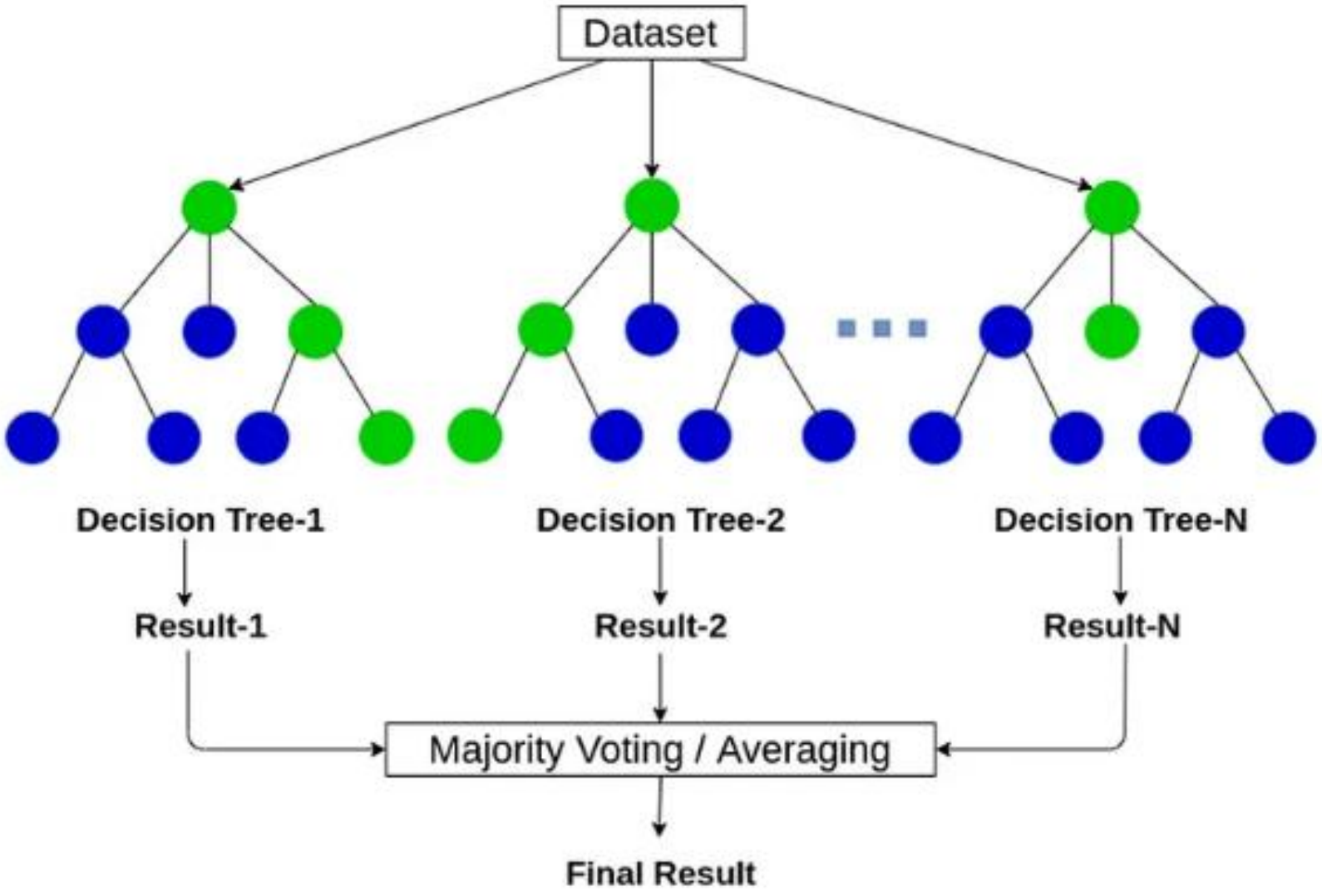
Как принимается решение?

Когда нужно сделать прогноз, каждое дерево в лесу дает свой «голос»:

В задачах классификации (например, «спам» или «не спам») побеждает вариант, за который проголосовало большинство деревьев.

В задачах регрессии (предсказание чисел, например, цены квартиры) результаты всех деревьев просто усредняются.

Случайные деревья и случайный лес



Преимущества и недостатки

Плюсы: Высокая точность, устойчивость к выбросам в данных и способность работать с большим количеством признаков без предварительной подготовки.

Минусы: Модель может занимать много памяти и работать медленнее, чем простые алгоритмы, из-за большого количества деревьев.